



APIs with Lambda + API Gateway



Kehinde Abiuwa

Create method

Method details

Method type
GET

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **HTTP**
Integrate with an existing HTTP endpoint.

☐ **Mock**
Generate a response based on API Gateway mappings and transformations.

☐ **AWS service**
Integrate with an AWS Service.

☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.

☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



Introducing Today's Project!

In this project, I will demonstrate how to build a serverless API using AWS Lambda and Amazon API Gateway—writing a Lambda function, creating an API Gateway endpoint, wiring them together, testing the endpoint, and documenting it in JSON. I'm doing this project to learn the core skills a backend/cloud engineer needs: designing stateless functions, configuring routes and methods, handling CORS and auth basics, deploying stages, and producing clear API documentation.

Tools and concepts

Services I used were AWS Lambda and Amazon API Gateway. Key concepts I learnt include Lambda functions (event-driven serverless code, handlers, and IAM permissions), API resources & methods (e.g., GET /users), Lambda proxy integration and request/response mapping, stages & deploys (stage URLs like /prod, throttling/timeouts), basic auth/errors (e.g., "Missing Authentication Token"), and publishing OpenAPI/Swagger documentation for the API.

Project reflection

This project took me approximately 60 minutes. The most challenging part was wiring up the GET method to Lambda and then remembering to deploy to a stage—I kept hitting the "Missing Authentication Token" error until I used the full stage + resource path and redeployed. I also had to be careful with IAM permissions and Lambda proxy integration so the request/response shapes matched. It was most rewarding to hit the stage URL and see Lambda return data through API Gateway, then publish and download the OpenAPI (Swagger) docs—felt like a complete, production-style serverless API from code to documentation.

I chose to do this project today because... Something that would make learning with NextWork even better is...



Lambda functions

AWS Lambda is a service that lets you run code without needing to manage any computers/servers - Lambda will manage them for you. Lambda runs your code only when you need it to (so you're not paying for any idle time). It also scales automatically, from a few requests per day to thousands per second - all you need to do is supply your code in one of the languages that Lambda supports. ChatGPT said: I'm using Lambda in this project to run the backend logic for my API—handle requests from API Gateway, fetch/return user data, and send clean JSON responses—without managing any servers. It's pay-per-use, scales automatically, and keeps the setup simple and low-cost.

The code I added to my function will set up a Lambda function that retrieves data from a DynamoDB table. It looks for specific user data based on a 'userId' and returns that data. If there's an error e.g. the userId doesn't exist in the database, it returns an error message.



Successfully created the function **RetrieveUserData**. You can now change its code and configuration. To invoke your function with a test event, choose "Test".

Code source Info

Open in Visual Studio Code

Upload from

RetrieveUserData

EXPLORER

RETRIEVEUSERDATA

JS index.mjs

DEPLOY [UNDEPLOYED CHANGES]

You have undeployed changes.

Deploy (⇧⌘U)

Test (⇧⌘T)

TEST EVENTS [NONE SELECTED]

Create new test event

JS index.mjs

```
1 // Import individual components from the DynamoDB client package
2 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
3 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dyna
4
5 const ddbCli = import DynamoDBDocumentClient n: 'eu-north-1' );
6 const ddb = DynamoDBDocumentClient.from(ddbCli);
7
8 async function handler(event) {
9   const userId = event.queryStringParameters.userId;
10   const params = {
11     TableName: 'UserData',
12     Key: { userId }
13   };
14
15   try {
16     const command = new GetCommand(params);
17     const { Item } = await ddb.send(command);
18     if (Item) {
19       return {
20         statusCode: 200,
21         headers: { 'Content-Type': 'application/json' },
22         body: JSON.stringify(Item)
23       };
24     }
25     return {
26       statusCode: 404,
27       headers: { 'Content-Type': 'application/json' },
28       body: JSON.stringify({ message: 'User not found' })
29     };
30   } catch (err) {
31     console.error('Error retrieving user data:', err);
32     return {
33       statusCode: 500,
34       headers: { 'Content-Type': 'application/json' },
35       body: JSON.stringify({ message: 'Internal server error' })
36     };
37   }
38 }
```

Info

Tutorials

Learn how to implement common use cases in AWS Lambda.

Create a simple web application

In this tutorial you will learn how to:

- Build a simple web application consisting of a Lambda function with a full URL that outputs a JSON response
- Invoke your function through its function URL

Learn more

Start tutorial

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Code

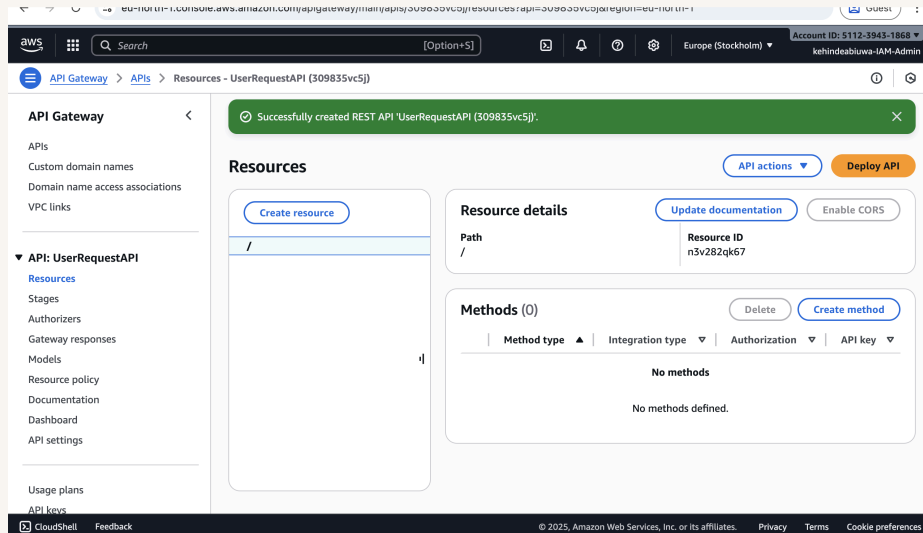


API Gateway

APIs are the rules/doors apps use to talk to each other—send a request, get a structured response. There are different types of APIs, like REST/HTTP (JSON over URLs), GraphQL (single endpoint with queries), WebSocket (real-time), gRPC (binary/fast), and SOAP (XML). My API is a simple REST-style HTTP API that returns JSON (e.g., GET /users/{id}), built with API Gateway in front and a Lambda function that does the work behind it.

API Gateway is AWS's "front door" for APIs—it receives HTTP requests, routes them to the right backend (like Lambda), handles auth, throttling, request/response mapping, CORS, and gives you tools to publish, monitor, and secure APIs at any scale. I'm using API Gateway in this project to expose my Lambda function as a public HTTP endpoint, validate and transform requests/responses (JSON), enable CORS, and control access/rate-limits—so the Lambda only runs when called through a managed, secure API.

When a user makes a request to my API Gateway URL, API Gateway first handles the front-door work—auth/CORS/throttling/validation—then converts the HTTP request into a JSON event and invokes my Lambda function. Lambda runs the code (e.g., look up user data), returns a response object (statusCode, headers, body). API Gateway maps that back to a normal HTTP response, adds any headers (like CORS), and sends it to the user





API Resources and Methods

An API is made up of resources, which are the distinct endpoints (paths) that handle a specific part of the API's functionality—like `/users`, `/users/{id}`, or `/messages`. Each resource groups related requests and responses, letting you attach methods (GET, POST, etc.), parameters, and permissions to that section of the API.

ChatGPT said: Each resource consists of methods, which are the specific actions you can perform on that resource using standard HTTP verbs. For example: GET (read data), POST (create data), PUT (update/replace data), and DELETE (remove data). Methods define how clients interact with your API's endpoints.

I created a GET method on my `/users` resource that's integrated with my Lambda function `RetrieveUserData` using Lambda proxy integration. This means when someone sends GET `/users`, API Gateway forwards the request to that Lambda, which fetches the data and returns a JSON response. I also granted API Gateway permission to invoke the Lambda so the call succeeds.



Create method

Method details

Method type

GET

Integration type

☒ **Lambda function**

Integrate your API with a Lambda function.



☐ **HTTP**

Integrate with an existing HTTP endpoint.



☐ **Mock**

Generate a response based on API Gateway mappings and transformations.



☐ **AWS service**

Integrate with an AWS Service.



☐ **VPC link**

Integrate with a resource that isn't accessible over the public internet.



☒ **Lambda proxy integration**

Send the request to your Lambda function as a structured event.

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

[Privacy](#)

[Terms](#)

[Cookie preferences](#)



API Deployment

When you deploy an API, you deploy it to a specific stage. A stage is a named snapshot of your API (with its own URL and settings like logging, caching, and throttling) that lets you separate environments—e.g., dev, test, and prod—and roll out changes safely. I deployed to the dev stage so I can test the endpoints before promoting them to production.

To visit my API, I copied the Invoke URL from the stage page and opened <https://309835vc5j.execute-api.eu-north-1.amazonaws.com/prod> in my browser. The API displayed an error because I haven't set up my DynamoDB table yet.

Deploy API

Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)

Stage

New stage

Stage name

prod

*A new stage will be created with the default settings. Edit your stage settings on the **Stage** page.*

Deployment description

Cancel Deploy



API Documentation

For my project's extension, I am writing API documentation because it tells other developers exactly how to use my API—its endpoints, methods, required parameters, example requests/responses, errors, and auth rules—so onboarding is faster, fewer mistakes happen, and the API is easier to maintain and test. You can do this in API Gateway by adding docs to your resources/methods and exporting an OpenAPI (Swagger) JSON spec (or by hand-writing an OpenAPI JSON file), then viewing it in tools like Swagger UI/Redoc or sharing a Postman collection.

Once I prepared my documentation, I can publish it to API Gateway → your API → Documentation → Publish documentation, pick the stage (prod), optionally add a version note, and click Publish. You have to publish your API to a specific stage because a stage is a snapshot of a deployed version of your API (e.g., /dev, /test, /prod). Tying docs to a stage guarantees that consumers see documentation that matches the behavior and invoke URL of that deployment (including the stage path), while letting you keep separate docs for different environments or versions.

My published and downloaded documentation showed me a Swagger 2.0 (OpenAPI) JSON for UserRequestAPI with a timestamped version, host of the execute-api domain, and a basePath of /prod over https. It listed one resource, /users, with a GET method that produces application/json and returns a 200 response (referencing an Empty schema). The GET is wired via an AWS proxy integration to my Lambda function RetrieveUserData (shown by the Lambda invocation ARN), with a 29-second timeout and CONVERT_TO_TEXT content handling. It also included the documentation block describing the API and its baseURL



```
10  "paths": {
11    "/users": {
12      "get": {
22        "x-amazon-apigateway-integration": {
31          "timeoutInMillis": 29000,
32          "contentHandling": "CONVERT_TO_TEXT",
33          "type": "aws_proxy"
34        }
35      }
36    },
37  },
38  "definitions": {
39    "Empty": {
40      "type": "object",
41      "title": "Empty Schema"
42    }
43  },
44  "x-amazon-apigateway-documentation": {
45    "version": "1",
46    "createdDate": "2025-09-04T18:28:09Z",
47    "documentationParts": [ {
48      "location": {
49        "type": "API"
50      },
51      "properties": {
52        "description": "The UserRequestAPI manages user data retrieval and manipulation. It supports operations to retrieve user details based on unique identifiers.",
53        "baseUrl": "https://309835vc5j.execute-api.eu-north-1.amazonaws.com/"
54      }
55    } ]
56  }
57 }
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

